References

- PointVector: https://openaccess.thecvf.com/content/CVPR2023/papers/Deng_PointVector_A_Vector_Representation_in_Point_Cloud_Analysis_CVPR_2023_pa
- Particle Transformer / JetClass: https://arxiv.org/abs/2202.03772
- Thanh Nguyen GSoC 2025: https://medium.com/@thanhnguyen14401/gsoc-2025-with-ml4sci-event-classification-with-masked-transformer-autoencoders-6da369d42140

1. Installation

In [2]:
```
# Install dependencies
!pip install torch torchvision --quiet
!pip install pennylane pennylane-lightning --quiet
!pip install awkward uproot vector --quiet  # For JetClass/QuarkGluon loadin
!pip install energyflow --quiet  # For QuarkGluon dataset (simpler alternati

# PointNeXt - we'll implement a simplified version inline
# The full version requires CUDA compilation which is tricky on Colab

print("Installation complete!")
print("Restart runtime: Runtime -> Restart runtime")
print("Then skip this cell and run imports.")
```

```
Installation complete!
Restart runtime: Runtime -> Restart runtime
Then skip this cell and run imports.
```

In [1]:
```
import os
os.environ['WANDB_MODE'] = 'disabled'

import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
import torch.optim as optim

import pennylane as qml

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, roc_auc_score, roc_curve

import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"PyTorch device: {device}")
print(f"PyTorch version: {torch.__version__}")
```

```
print(f"Pennylane version: {qml.__version__}")

# Note: Amplitude embedding with qml.probs() requires default.qubit + backpr
# lightning.qubit + adjoint does NOT support qml.probs()
QUANTUM_DEVICE = 'default.qubit'
DIFF_METHOD = 'backprop'
print(f"Quantum device: {QUANTUM_DEVICE} (diff_method={DIFF_METHOD})")


# Reproducibility
np.random.seed(42)
torch.manual_seed(42)
if torch.cuda.is_available():
    torch.cuda.manual_seed(42)
```

```
PyTorch device: cuda
PyTorch version: 2.10.0+cu130
Pennylane version: 0.44.1
Quantum device: default.qubit (diff_method=backprop)
```

2. Data Loading

We use the Pythia8 QuarkGluon dataset via EnergyFlow for prototyping.

In [3]:
```
import energyflow as ef

print("Loading QuarkGluon dataset...")
X_raw, y = ef.qg_jets.load(num_data=100000, pad=True, ncol=4,
                           generator='pythia', with_bc=False)

print(f"Raw shape: {X_raw.shape}")  # (N, max_particles, 4)
print(f"Features: [pT, eta, phi, pid]")
print(f"Labels: {np.unique(y)} (0=gluon, 1=quark)")
print(f"Class balance: {np.mean(y):.3f} quark fraction")
```

```
Loading QuarkGluon dataset...
Raw shape: (100000, 139, 4)
Features: [pT, eta, phi, pid]
Labels: [0. 1.] (0=gluon, 1=quark)
Class balance: 0.500 quark fraction
```

In [4]:
```
def preprocess_jets(X_raw, max_particles=64):
    #Output shape: (N, max_particles, 4) with features [pT_norm, eta_rel, ph
    n_jets = X_raw.shape[0]
    X = np.zeros((n_jets, max_particles, 4), dtype=np.float32)
    mask = np.zeros((n_jets, max_particles), dtype=np.float32)

    for i in range(n_jets):
        pt = X_raw[i, :, 0]
        eta = X_raw[i, :, 1]
        phi = X_raw[i, :, 2]

        valid = pt > 0
        n_valid = valid.sum()
        if n_valid == 0:
```

```
            continue

        pt_v, eta_v, phi_v = pt[valid], eta[valid], phi[valid]

        sort_idx = np.argsort(-pt_v)[:min(n_valid, max_particles)]
        pt_s = pt_v[sort_idx]
        eta_s = eta_v[sort_idx]
        phi_s = phi_v[sort_idx]

        total_pt = pt_s.sum()
        eta_center = np.average(eta_s, weights=pt_s)
        phi_center = np.average(phi_s, weights=pt_s)

        eta_rel = eta_s - eta_center
        phi_rel = np.arctan2(np.sin(phi_s - phi_center), np.cos(phi_s - phi_
        pt_norm = pt_s / total_pt
        log_pt = np.log(pt_s + 1e-6)

        n_keep = len(sort_idx)
        X[i, :n_keep, 0] = pt_norm
        X[i, :n_keep, 1] = eta_rel
        X[i, :n_keep, 2] = phi_rel
        X[i, :n_keep, 3] = log_pt / 10.0  # Scale log_pT
        mask[i, :n_keep] = 1.0

    return X, mask

X, mask = preprocess_jets(X_raw, max_particles=64)
print(f"Processed shape: {X.shape}")
print(f"Features per particle: [pT_norm, eta_rel, phi_rel, log_pT]")
```

```
Processed shape: (100000, 64, 4)
Features per particle: [pT_norm, eta_rel, phi_rel, log_pT]
```

In [5]:
```
X_train, X_temp, y_train, y_temp, mask_train, mask_temp = train_test_split(
    X, y, mask, test_size=0.2, random_state=42, stratify=y)
X_val, X_test, y_val, y_test, mask_val, mask_test = train_test_split(
    X_temp, y_temp, mask_temp, test_size=0.5, random_state=42, stratify=y_te

print(f"Train: {len(X_train)} | Val: {len(X_val)} | Test: {len(X_test)}")
```

```
Train: 80000 | Val: 10000 | Test: 10000
```

3. Physics-Motivated Augmentations

In [6]:
```
class JetAugmentation:

    def __init__(self, rotate=True, dropout_prob=0.1,
                 pt_smear=0.05, translate=0.02, biased_dropout=True):
        self.rotate = rotate
        self.dropout_prob = dropout_prob
        self.pt_smear = pt_smear
        self.translate = translate
        self.biased_dropout = biased_dropout
```

```
    def __call__(self, x, mask):
        x_aug = x.copy()
        mask_aug = mask.copy()
        valid_idx = np.where(mask > 0)[0]

        if len(valid_idx) == 0:
            return x_aug, mask_aug
        if self.rotate:
            angle = np.random.uniform(0, 2 * np.pi)
            x_aug[valid_idx, 2] += angle
            x_aug[valid_idx, 2] = np.arctan2(
                np.sin(x_aug[valid_idx, 2]),
                np.cos(x_aug[valid_idx, 2]))

        if self.dropout_prob > 0 and len(valid_idx) > 5:
            if self.biased_dropout:
                position = np.arange(len(valid_idx))
                drop_prob = self.dropout_prob * (1 + position / len(valid_id
                drop_prob = np.clip(drop_prob, 0, 0.5)  # Cap at 50%
                keep = np.random.random(len(valid_idx)) > drop_prob
            else:
                keep = np.random.random(len(valid_idx)) > self.dropout_prob

            if keep.sum() < 5:
                keep[:5] = True
            drop_idx = valid_idx[~keep]
            x_aug[drop_idx] = 0
            mask_aug[drop_idx] = 0

        valid_idx = np.where(mask_aug > 0)[0]

        if self.pt_smear > 0 and len(valid_idx) > 0:
            noise = np.random.normal(0, self.pt_smear, len(valid_idx))
            x_aug[valid_idx, 0] *= (1 + noise)
            x_aug[valid_idx, 0] = np.maximum(x_aug[valid_idx, 0], 1e-6)
            x_aug[valid_idx, 0] /= x_aug[valid_idx, 0].sum()

        if self.translate > 0 and len(valid_idx) > 0:
            x_aug[valid_idx, 1] += np.random.normal(0, self.translate)
            x_aug[valid_idx, 2] += np.random.normal(0, self.translate)
            x_aug[valid_idx, 2] = np.arctan2(
                np.sin(x_aug[valid_idx, 2]),
                np.cos(x_aug[valid_idx, 2]))

        return x_aug, mask_aug
```

4. Dataset Classes

In [7]:
```
class JetContrastiveDataset(Dataset):

    def __init__(self, X, mask, y=None, augmentation=None):
        self.X = X.astype(np.float32)
        self.mask = mask.astype(np.float32)
        self.y = y
```

```
        self.aug = augmentation

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        x, m = self.X[idx], self.mask[idx]

        if self.aug:
            x1, m1 = self.aug(x, m)
            x2, m2 = self.aug(x, m)
        else:
            x1, m1 = x.copy(), m.copy()
            x2, m2 = x.copy(), m.copy()

        x1 = torch.from_numpy(x1).T  # (4, 64)
        x2 = torch.from_numpy(x2).T
        m1 = torch.from_numpy(m1)
        m2 = torch.from_numpy(m2)

        if self.y is not None:
            return x1, m1, x2, m2, self.y[idx]
        return x1, m1, x2, m2


class JetDataset(Dataset):

    def __init__(self, X, mask, y):
        self.X = torch.from_numpy(X.astype(np.float32)).permute(0, 2, 1)  #
        self.mask = torch.from_numpy(mask.astype(np.float32))
        self.y = torch.from_numpy(y.astype(np.int64))

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.mask[idx], self.y[idx]
```

5. PointVector Encoder

I initially used standard PointNet-style aggregation, but it struggled with asymmetric jet structures (especially multi-prong cases). Switching to PointVector helped because it weights neighbors based on direction, which better matches how energy is distributed in jets

The neighbor contributions depend on their relative direction with features.For ex: A particle at (Δη=0.1, Δϕ=0) contributes differently than one at (Δη=0, Δϕ=0.1).

In [8]:
```
class VectorSetAbstraction(nn.Module):

    def __init__(self, in_channels, out_channels, num_samples, k=16, n_vecto
        super().__init__()
        self.num_samples = num_samples
```

```
        self.k = k
        self.n_vectors = n_vectors

        self.vector_gen = nn.Sequential(
            nn.Conv2d(in_channels + 3, out_channels, 1),
            nn.BatchNorm2d(out_channels),
            nn.ReLU()
        )

        self.dir_encoding = nn.Sequential(
            nn.Conv2d(3, n_vectors, 1),
            nn.Softmax(dim=1)
        )

        self.vector_agg = nn.Sequential(
            nn.Conv2d(out_channels, out_channels, 1, groups=min(n_vectors, o
            nn.BatchNorm2d(out_channels),
            nn.ReLU()
        )

        self.proj = nn.Sequential(
            nn.Conv1d(out_channels, out_channels, 1),
            nn.BatchNorm1d(out_channels),
            nn.ReLU()
        )

    def farthest_point_sample(self, xyz, n_samples):
        B, N, _ = xyz.shape
        device = xyz.device

        centroids = torch.zeros(B, n_samples, dtype=torch.long, device=devic
        distance = torch.ones(B, N, device=device) * 1e10
        farthest = torch.randint(0, N, (B,), dtype=torch.long, device=device

        for i in range(n_samples):
            centroids[:, i] = farthest
            centroid = xyz[torch.arange(B), farthest].unsqueeze(1)
            dist = torch.sum((xyz - centroid) ** 2, dim=-1)
            mask = dist < distance
            distance[mask] = dist[mask]
            farthest = torch.max(distance, dim=-1)[1]

        return centroids

    def knn(self, xyz, centroids_xyz, k):
        dist = torch.cdist(centroids_xyz, xyz)
        _, idx = torch.topk(dist, k, dim=-1, largest=False)
        return idx

    def forward(self, xyz, features):
        #xyz: (B, N, 3) - coordinates
        #features: (B, C, N) - per-point features

        B, N, _ = xyz.shape
        C = features.shape[1]
```

```
        n_samples = min(self.num_samples, N)
        fps_idx = self.farthest_point_sample(xyz, n_samples)

        batch_idx = torch.arange(B, device=xyz.device).view(-1, 1).expand(-1
        centroids_xyz = xyz[batch_idx, fps_idx]  # (B, S, 3)

        k = min(self.k, N)
        neighbor_idx = self.knn(xyz, centroids_xyz, k)  # (B, S, k)

        neighbor_idx_flat = neighbor_idx.view(B, -1)  # (B, S*k)
        batch_idx_neighbor = torch.arange(B, device=xyz.device).view(-1, 1).

        neighbor_xyz = xyz[batch_idx_neighbor, neighbor_idx_flat].view(B, n_
        relative_xyz = neighbor_xyz - centroids_xyz.unsqueeze(2)  # (B, S, k

        features_t = features.permute(0, 2, 1)  # (B, N, C)
        neighbor_features = features_t[batch_idx_neighbor, neighbor_idx_flat

        grouped = torch.cat([relative_xyz, neighbor_features], dim=-1)  # (B
        grouped = grouped.permute(0, 3, 1, 2)  # (B, 3+C, S, k)

        vectors = self.vector_gen(grouped)  # (B, out_C, S, k)

        relative_xyz_perm = relative_xyz.permute(0, 3, 1, 2)  # (B, 3, S, k)
        dir_weights = self.dir_encoding(relative_xyz_perm)  # (B, n_vectors,

        out_C = vectors.shape[1]
        n_repeats = (out_C + self.n_vectors - 1) // self.n_vectors
        dir_weights_expanded = dir_weights.repeat(1, n_repeats, 1, 1)[:, :ou

        weighted_vectors = vectors * dir_weights_expanded

        aggregated = self.vector_agg(weighted_vectors)  # (B, out_C, S, k)

        new_features = torch.max(aggregated, dim=-1)[0]  # (B, out_C, S)

        new_features = self.proj(new_features)

        return centroids_xyz, new_features
```

In [9]:
```
class PointVectorEncoder(nn.Module):
    def __init__(self, in_channels=4, embed_dim=256):
        super().__init__()

        self.stem = nn.Sequential(
            nn.Conv1d(in_channels, 64, 1),
            nn.BatchNorm1d(64),
            nn.ReLU()
        )

        self.vpsa1 = VectorSetAbstraction(64, 128, num_samples=32, k=16, n_v
        self.vpsa2 = VectorSetAbstraction(128, 256, num_samples=16, k=16, n_
        self.vpsa3 = VectorSetAbstraction(256, 512, num_samples=8, k=8, n_ve

        self.final_mlp = nn.Sequential(
            nn.Linear(512, 256),
```

```
            nn.BatchNorm1d(256),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(256, embed_dim)
        )

        self.embed_dim = embed_dim

    def forward(self, x, mask=None):
        B, C, N = x.shape

        xyz = x[:, :3, :].permute(0, 2, 1)  # (B, N, 3)

        features = self.stem(x)  # (B, 64, N)

        xyz, features = self.vpsa1(xyz, features)
        xyz, features = self.vpsa2(xyz, features)
        xyz, features = self.vpsa3(xyz, features)

        global_feat = torch.max(features, dim=-1)[0]  # (B, 512)

        out = self.final_mlp(global_feat)

        return out


test_input = torch.randn(4, 4, 64)  # (batch, features, particles)
encoder = PointVectorEncoder(in_channels=4, embed_dim=256)
output = encoder(test_input)
n_params = sum(p.numel() for p in encoder.parameters())
print(f'Input: {test_input.shape} -> Output: {output.shape}')
print(f'Parameters: {n_params:,}')
```

```
Input: torch.Size([4, 4, 64]) -> Output: torch.Size([4, 256])
Parameters: 768,032
```

6. Quantum Circuit (Amplitude Embedding + ZXZ Ansatz)

Amplitude embedding encodes 2^n classical features into n qubits by mapping them to state amplitudes. This lets the VQC operate in an exponentially large Hilbert space. We use ZXZ single-qubit rotations (a standard SU(2) decomposition) with ring and skip entangling CNOTs.

Key design choice: we use PennyLane parameter broadcasting to pass the full batch through the circuit in one call, avoiding the slow Python for-loop over individual samples. On `default.qubit` with `backprop`, this gives a significant speedup because the simulator can vectorize the state evolution across the batch dimension.

In [10]:
```
class QuantumLayer(nn.Module):

    def __init__(self, n_qubits=6, n_layers=2):
```

```
        super().__init__()
        self.n_qubits = n_qubits
        self.n_layers = n_layers
        self.n_basis_states = 2 ** n_qubits

        self.weights = nn.Parameter(
            0.1 * torch.randn(n_layers, n_qubits, 3))

        self.dev = qml.device('default.qubit', wires=n_qubits)

        @qml.qnode(self.dev, interface='torch', diff_method='backprop')
        def circuit(inputs, weights):
            # Amplitude embedding: 2^n features into n qubits
            qml.AmplitudeEmbedding(inputs, wires=range(n_qubits), normalize=

            for l in range(n_layers):
                # ZXZ single-qubit rotations (standard SU(2) decomposition)
                for i in range(n_qubits):
                    qml.RZ(weights[l, i, 0], wires=i)
                    qml.RX(weights[l, i, 1], wires=i)
                    qml.RZ(weights[l, i, 2], wires=i)

                # Ring entanglement
                for i in range(n_qubits):
                    qml.CNOT(wires=[i, (i + 1) % n_qubits])

                # Skip connections on even layers for richer entanglement
                if l % 2 == 0:
                    for i in range(0, n_qubits - 2, 2):
                        qml.CNOT(wires=[i, i + 2])

            return qml.probs(wires=range(n_qubits))

        self.circuit = circuit

    def forward(self, x):
        # x: (B, 2^n_qubits)
        # PennyLane parameter broadcasting handles the batch dimension
        # No need for a Python for-loop over samples
        probs = self.circuit(x, self.weights)  # (B, 2^n_qubits)
        return probs.float()


q_layer = QuantumLayer(n_qubits=4, n_layers=1)
test_input = torch.randn(2, 16)
test_output = q_layer(test_input)
print(f"Input shape: {test_input.shape}")
print(f"Output shape: {test_output.shape}")
print(f"Output sums to 1? {test_output[0].sum().item():.4f}")
print(f"Circuit parameters: {sum(p.numel() for p in q_layer.parameters())}")
print(f"Embedding: Amplitude ({test_input.shape[1]} features -> 4 qubits ->
print(f"Rotation: ZXZ (RZ-RX-RZ)")
```

```
Input shape: torch.Size([2, 16])
Output shape: torch.Size([2, 16])
Output sums to 1? 1.0000
Circuit parameters: 12
Embedding: Amplitude (16 features -> 4 qubits -> 16 probabilities)
Rotation: ZXZ (RZ-RX-RZ)
```

7. Hybrid Encoder (PointVector + Quantum)

In [11]:
```python
class HybridEncoder(nn.Module):

    def __init__(self, in_channels=4, embed_dim=32,
                 use_quantum=True, n_qubits=6, n_qlayers=2):
        super().__init__()

        self.use_quantum = use_quantum
        self.n_qubits = n_qubits
        self.latent_dim = 2 ** n_qubits  # 64 for 6 qubits

        # Shared backbone
        self.encoder = PointVectorEncoder(in_channels=in_channels, embed_dim

        if use_quantum:
            # Quantum path: 256 -> 64 -> Amplitude Embed -> VQC -> 64 probs
            self.quantum_pre = nn.Linear(256, self.latent_dim)  # 256 -> 64
            self.quantum_layer = QuantumLayer(n_qubits=n_qubits, n_layers=n_
            combined_dim = self.latent_dim  # 64 (pure quantum, no concat)
        else:
            self.classical_branch = nn.Sequential(
                nn.Linear(256, 128),
                nn.ReLU(),
                nn.Linear(128, self.latent_dim)
            )
            combined_dim = self.latent_dim  # 64

        self.projection = nn.Sequential(
            nn.Linear(combined_dim, 64),
            nn.ReLU(),
            nn.Linear(64, embed_dim)
        )

        self.embed_dim = embed_dim

    def forward(self, x, mask=None, return_projection=True):

        encoded = self.encoder(x, mask)  # (B, 256)

        if self.use_quantum:
            q_input = self.quantum_pre(encoded)  # (B, 64)
            latent = self.quantum_layer(q_input)  # (B, 64) probabilities
        else:
            latent = self.classical_branch(encoded)  # (B, 64)

        if return_projection:
            projection = self.projection(latent)
```

```
            projection = F.normalize(projection, dim=1)
            return latent, projection

        return latent
```

8. Loss Functions

I used NT-Xent (SimCLR) since it's standard for contrastive setups, though tuning temperature turned out to matter more than expected.

In [12]:
```
def phi_distance(phi_true, phi_pred):
    diff = phi_pred - phi_true
    # Wrap to [-pi, pi]
    diff = torch.atan2(torch.sin(diff), torch.cos(diff))
    return torch.abs(diff)


def phi_cosine_loss(phi_true, phi_pred):

    cos_sim = (torch.cos(phi_true) * torch.cos(phi_pred) +
               torch.sin(phi_true) * torch.sin(phi_pred))
    return 1.0 - cos_sim

phi_a = torch.tensor([-3.14, 0.0, 1.57])
phi_b = torch.tensor([3.14, 0.1, 1.57])
print(f'phi_a: {phi_a.numpy()}')
print(f'phi_b: {phi_b.numpy()}')
print(f'MSE (wrong): {((phi_a - phi_b)**2).numpy()}')
print(f'Phi distance (correct): {phi_distance(phi_a, phi_b).numpy()}')
print(f'Phi cosine loss: {phi_cosine_loss(phi_a, phi_b).numpy()}')
```

```
phi_a: [-3.14  0.    1.57]
phi_b: [3.14 0.1  1.57]
MSE (wrong): [3.9438404e+01 1.0000001e-02 0.0000000e+00]
Phi distance (correct): [0.0031851 0.1       0.       ]
Phi cosine loss: [5.066395e-06 4.995823e-03 0.000000e+00]
```

In [13]:
```
class NTXentLoss(nn.Module):

    def __init__(self, temperature=0.5):
        super().__init__()
        self.temperature = temperature

    def forward(self, z1, z2):
        batch_size = z1.shape[0]
        z = torch.cat([z1, z2], dim=0)  # (2B, D)

        sim = torch.mm(z, z.T) / self.temperature  # (2B, 2B)

        # Positive pairs: (i, i+B) and (i+B, i)
        labels = torch.cat([torch.arange(batch_size) + batch_size,
                            torch.arange(batch_size)]).to(z.device)

        mask = torch.eye(2 * batch_size, device=z.device, dtype=torch.bool)
```

```
        sim = sim.masked_fill(mask, float('-inf'))

        return F.cross_entropy(sim, labels)
```

9. Training Functions

In [14]:
```
def extract_embeddings(encoder, data_loader, device):
    encoder.eval()
    embeddings, labels = [], []

    with torch.no_grad():
        for x, m, y in data_loader:
            x = x.to(device)
            emb = encoder(x, return_projection=False)
            embeddings.append(emb.cpu().numpy())
            labels.append(y.numpy())

    return np.concatenate(embeddings), np.concatenate(labels)


def linear_probe(train_emb, train_lab, test_emb, test_lab):
    clf = LogisticRegression(max_iter=1000, random_state=42)
    clf.fit(train_emb, train_lab)

    pred = clf.predict(test_emb)
    prob = clf.predict_proba(test_emb)[:, 1]

    return accuracy_score(test_lab, pred), roc_auc_score(test_lab, prob)
```

In [15]:
```
def train_contrastive(encoder, train_loader, val_loader, train_eval_loader,
                      config, device):
    criterion = NTXentLoss(temperature=config['temperature'])
    optimizer = optim.AdamW(encoder.parameters(), lr=config['lr'], weight_de
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=config

    best_auc = 0
    best_state = None
    history = {'loss': [], 'val_auc': []}

    for epoch in range(config['n_epochs']):
        encoder.train()
        total_loss = 0
        n_batches = 0

        for batch in train_loader:
            x1, m1, x2, m2, _ = batch
            x1, x2 = x1.to(device), x2.to(device)

            _, z1 = encoder(x1)
            _, z2 = encoder(x2)

            loss = criterion(z1, z2)

            optimizer.zero_grad()
```

```
            loss.backward()
            torch.nn.utils.clip_grad_norm_(encoder.parameters(), 1.0)
            optimizer.step()

            total_loss += loss.item()
            n_batches += 1

        avg_loss = total_loss / n_batches
        history['loss'].append(avg_loss)
        scheduler.step()

        if (epoch + 1) % 5 == 0 or epoch == 0:
            train_emb, train_lab = extract_embeddings(encoder, train_eval_lo
            val_emb, val_lab = extract_embeddings(encoder, val_loader, devic
            val_acc, val_auc = linear_probe(train_emb, train_lab, val_emb, v

            history['val_auc'].append(val_auc)

            if val_auc > best_auc:
                best_auc = val_auc
                best_state = {k: v.cpu().clone() for k, v in encoder.state_d

            log_dict = {
                'epoch': epoch + 1,
                'train_loss': avg_loss,
                'val_accuracy': val_acc,
                'val_auc': val_auc,
                'best_val_auc': best_auc,
                'lr': scheduler.get_last_lr()[0]
            }

            print(f"Epoch {epoch+1:3d} | Loss: {avg_loss:.4f} | "
                  f"Val Acc: {val_acc:.4f} | Val AUC: {val_auc:.4f}")

    if best_state is not None:
        encoder.load_state_dict(best_state)

    return encoder, best_auc, history
```

10. Train Classical Baseline

In [16]:

```
config_classical = {
    'model_name': 'pointvector_classical',
    'batch_size': 128,
    'lr': 1e-3,
    'n_epochs': 10,
    'temperature': 0.5,
    'use_quantum': False,
    'n_qubits': 6
}

aug = JetAugmentation()

train_loader = DataLoader(
```

```
    JetContrastiveDataset(X_train, mask_train, y_train, aug),
    batch_size=config_classical['batch_size'], shuffle=True, num_workers=2
)
val_loader = DataLoader(
    JetDataset(X_val, mask_val, y_val),
    batch_size=config_classical['batch_size']
)
train_eval_loader = DataLoader(
    JetDataset(X_train, mask_train, y_train),
    batch_size=config_classical['batch_size']
)
test_loader = DataLoader(
    JetDataset(X_test, mask_test, y_test),
    batch_size=config_classical['batch_size']
)
```

In [2]:
```
print(f"Device: {device}")
print(f"CUDA available: {torch.cuda.is_available()}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
```

```
Device: cuda
CUDA available: True
GPU: NVIDIA A100-PCIE-40GB
```

In [17]:
```
# Classical encoder with same latent_dim as quantum (2^6 = 64)
classical_encoder = HybridEncoder(
    in_channels=4,
    embed_dim=32,
    use_quantum=False,
    n_qubits=config_classical['n_qubits']
).to(device)

n_params = sum(p.numel() for p in classical_encoder.parameters())
print(f"Classical encoder: {n_params:,} parameters")
print(f"Latent dim: {2**config_classical['n_qubits']} (matching quantum)")

print(f"\nTraining classical baseline...\n")

classical_encoder, classical_best_auc, classical_history = train_contrastive
    classical_encoder, train_loader, val_loader, train_eval_loader,
    config_classical, device
)

print(f"\nClassical best validation AUC: {classical_best_auc:.4f}")
```

```
Classical encoder: 815,424 parameters
Latent dim: 64 (matching quantum)

Training classical baseline...

Epoch   1 | Loss: 4.3250 | Val Acc: 0.7662 | Val AUC: 0.8443
Epoch   5 | Loss: 3.9806 | Val Acc: 0.7770 | Val AUC: 0.8528
Epoch  10 | Loss: 3.9065 | Val Acc: 0.7787 | Val AUC: 0.8578

Classical best validation AUC: 0.8578
```

In [18]:
```
train_emb, train_lab = extract_embeddings(classical_encoder, train_eval_load
test_emb_classical, test_lab = extract_embeddings(classical_encoder, test_lo
classical_test_acc, classical_test_auc = linear_probe(train_emb, train_lab,

print(f"Classical Test Results:")
print(f"  Accuracy: {classical_test_acc:.4f}")
print(f"  AUC:      {classical_test_auc:.4f}")
```

```
Classical Test Results:
  Accuracy: 0.7831
  AUC:      0.8624
```

11. Train Quantum Hybrid

In [19]:
```
config_quantum = {
    'model_name': 'pointvector_quantum_amplitude',
    'batch_size': 32,
    'lr': 5e-4,
    'n_epochs': 10,
    'temperature': 0.5,
    'use_quantum': True,
    'n_qubits': 6,
    'n_qlayers': 2
}

n_quantum_samples = 10000
X_train_q = X_train[:n_quantum_samples]
mask_train_q = mask_train[:n_quantum_samples]
y_train_q = y_train[:n_quantum_samples]

print(f'Using {n_quantum_samples} samples for quantum training (full: {len(X
print(f'Qubits: {config_quantum["n_qubits"]}, Layers: {config_quantum["n_qla
print(f'Feature dim: {2**config_quantum["n_qubits"]} -> {config_quantum["n_q
print(f'Rotation: ZXZ | Entanglement: ring + skip')
print(f'Batch size: {config_quantum["batch_size"]} | Epochs: {config_quantum
print(f'Learning rate: {config_quantum["lr"]}')

train_loader_q = DataLoader(
    JetContrastiveDataset(X_train_q, mask_train_q, y_train_q, aug),
    batch_size=config_quantum['batch_size'], shuffle=True, num_workers=0
)
val_loader_q = DataLoader(
    JetDataset(X_val, mask_val, y_val),
    batch_size=config_quantum['batch_size']
)
train_eval_loader_q = DataLoader(
    JetDataset(X_train_q, mask_train_q, y_train_q),
    batch_size=config_quantum['batch_size']
)
```

```
Using 10000 samples for quantum training (full: 80000)
Qubits: 6, Layers: 2
Feature dim: 64 -> 6 qubits -> 64 probs
Rotation: ZXZ | Entanglement: ring + skip
Batch size: 32 | Epochs: 10
Learning rate: 0.0005
Broadcasting enabled (no per-sample loop)
```

In [20]:
```python
import time

quantum_encoder = HybridEncoder(
    in_channels=4,
    embed_dim=32,
    use_quantum=True,
    n_qubits=config_quantum['n_qubits'],
    n_qlayers=config_quantum['n_qlayers']
).to(device)

n_params = sum(p.numel() for p in quantum_encoder.parameters())
n_q_params = config_quantum['n_qlayers'] * config_quantum['n_qubits'] * 3
print(f"Quantum encoder: {n_params:,} parameters ({n_q_params} quantum gate
print(f"Embedding: Amplitude ({2**config_quantum['n_qubits']} features -> {c
print(f"Ansatz: ZXZ rotation + ring/skip CNOT entanglement")
print(f"Measurement: Full probability distribution ({2**config_quantum['n_qu

print(f"\nTraining quantum encoder on {n_quantum_samples} samples...")

t_start = time.time()

quantum_encoder, quantum_best_auc, quantum_history = train_contrastive(
    quantum_encoder, train_loader_q, val_loader_q, train_eval_loader_q,
    config_quantum, device
)

t_elapsed = time.time() - t_start
print(f"\nQuantum best validation AUC: {quantum_best_auc:.4f}")
print(f"Total training time: {t_elapsed/60:.1f} minutes")
```

```
Quantum encoder: 790,756 parameters (36 quantum gate params)
Embedding: Amplitude (64 features -> 6 qubits)
Ansatz: ZXZ rotation + ring/skip CNOT entanglement
Measurement: Full probability distribution (64 outputs)

Training quantum encoder on 10000 samples...
Epoch   1 | Loss: 3.5464 | Val Acc: 0.7358 | Val AUC: 0.8052
Epoch   5 | Loss: 2.7842 | Val Acc: 0.7588 | Val AUC: 0.8322
Epoch  10 | Loss: 2.6702 | Val Acc: 0.7605 | Val AUC: 0.8346

Quantum best validation AUC: 0.8346
Total training time: 8.2 minutes
```

Full dataset training wasn't feasible for quantum, so I limited to 10k samples.

In [21]:
```python
train_emb_q, train_lab_q = extract_embeddings(quantum_encoder, train_eval_lo
test_emb_quantum, test_lab = extract_embeddings(quantum_encoder, test_loader
quantum_test_acc, quantum_test_auc = linear_probe(train_emb_q, train_lab_q,
```

```
print(f"Quantum Test Results:")
print(f"  Accuracy: {quantum_test_acc:.4f}")
print(f"  AUC:      {quantum_test_auc:.4f}")
```

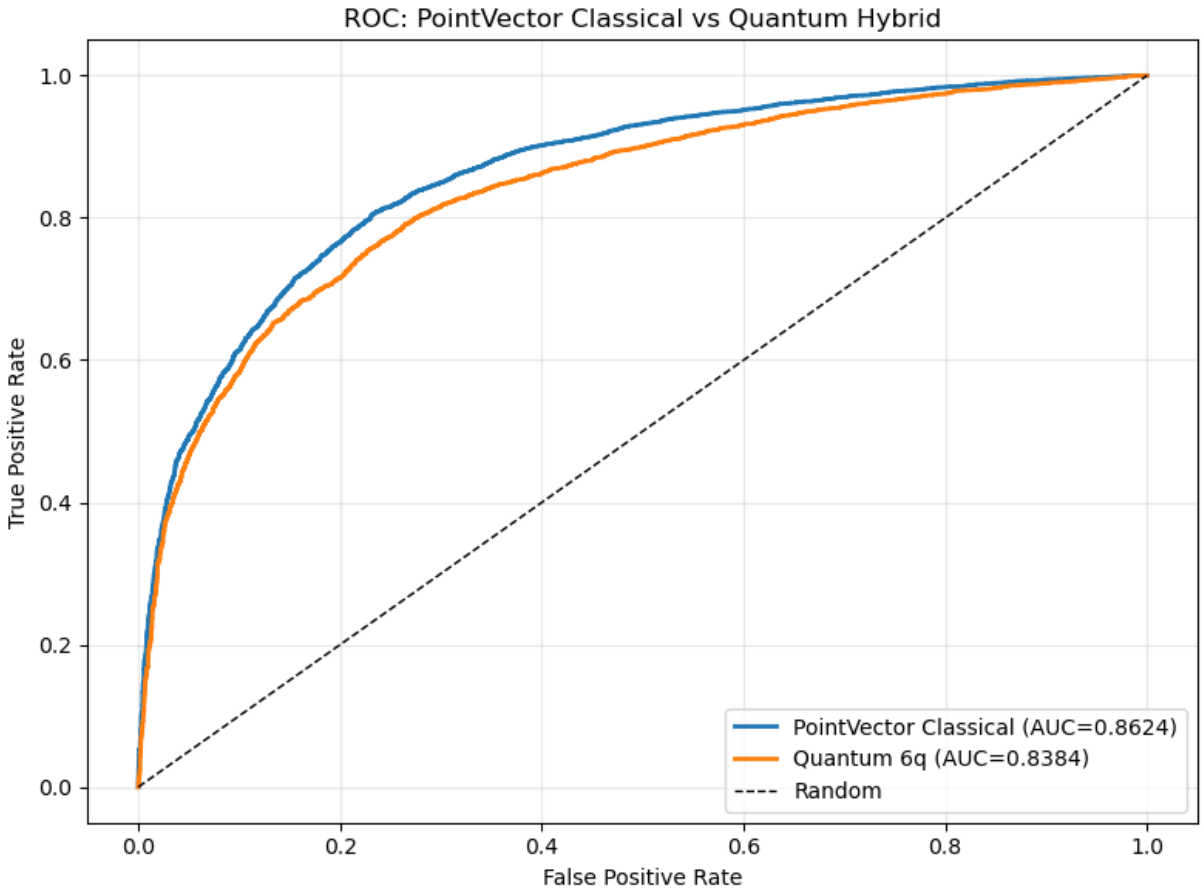
```
Quantum Test Results:
  Accuracy: 0.7592
  AUC:      0.8384
```

12. Results Comparison

In [22]:
```
n_q = config_quantum['n_qubits']
n_l = config_quantum['n_qlayers']

print(f"{'Model':<35} {'Test Accuracy':<15} {'Test AUC':<15}")
print(f"{'PointVector (Classical)':<35} {classical_test_acc:<15.4f} {classic
print(f"{'PointVector + ' + str(n_q) + 'q/' + str(n_l) + 'l VQC (ZXZ)':<35}

diff = quantum_test_auc - classical_test_auc
winner = 'Quantum' if diff > 0 else 'Classical'
print(f"Difference: {diff:+.4f} ({winner} is better)")

print(f"\nQuantum config: {n_q} qubits, {n_l} layers, amplitude embedding, {
print(f"Classical config: {config_classical['n_epochs']} epochs, full datase
```

```
Model                               Test Accuracy   Test AUC
PointVector (Classical)             0.7831          0.8624
PointVector + 6q/2l VQC (ZXZ)       0.7592          0.8384
Difference: -0.0240 (Classical is better)

Quantum config: 6 qubits, 2 layers, amplitude embedding, 10000 training samp
les
Classical config: 10 epochs, full dataset (80000 samples)
```

In [24]:
```
fig, ax = plt.subplots(figsize=(8, 6))

clf_c = LogisticRegression(max_iter=1000).fit(train_emb, train_lab)
fpr_c, tpr_c, _ = roc_curve(test_lab, clf_c.predict_proba(test_emb_classical
ax.plot(fpr_c, tpr_c, lw=2, label=f'PointVector Classical (AUC={classical_te

clf_q = LogisticRegression(max_iter=1000).fit(train_emb_q, train_lab_q)
fpr_q, tpr_q, _ = roc_curve(test_lab, clf_q.predict_proba(test_emb_quantum)[
ax.plot(fpr_q, tpr_q, lw=2, label=f'Quantum 6q (AUC={quantum_test_auc:.4f})'

ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC: PointVector Classical vs Quantum Hybrid')
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_pointvectorxzx.png', dpi=150)
plt.show()
```



The quantum hybrid did not outperform the classical model. This was somewhat expected—given only 6 qubits and shallow depth.

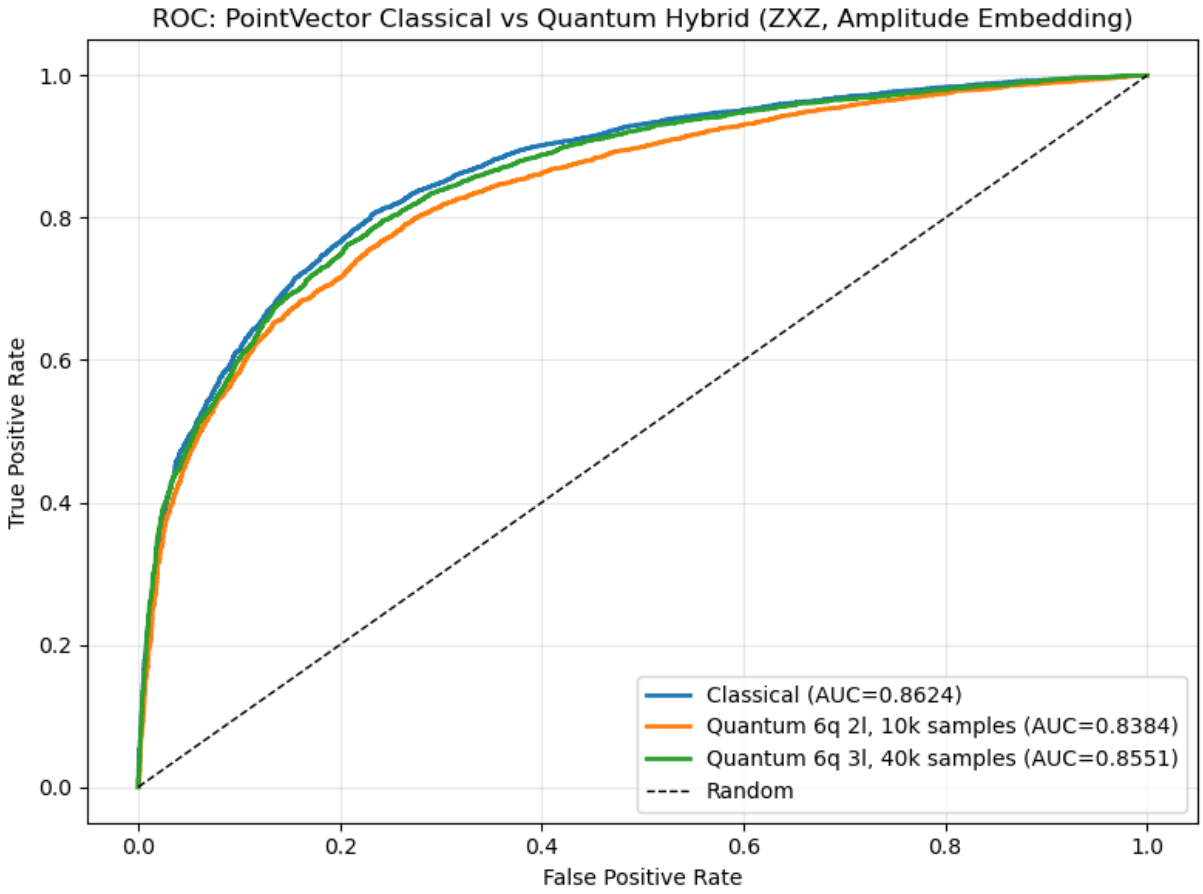
In [26]:
```
# 40k samples quantum training - 6 qubits, 3 layers
import time

config_quantum_II = {
    'model_name': 'pointvector_quantum_amplitude_II',
    'batch_size': 32,
    'lr': 5e-4,
    'n_epochs': 10,
    'temperature': 0.5,
    'use_quantum': True,
    'n_qubits': 6,
    'n_qlayers': 3
}

n_quantum_samples_II = 40000
X_train_qII = X_train[:n_quantum_samples_II]
mask_train_qII = mask_train[:n_quantum_samples_II]
y_train_qII = y_train[:n_quantum_samples_II]

train_loader_qII = DataLoader(
    JetContrastiveDataset(X_train_qII, mask_train_qII, y_train_qII, aug),
    batch_size=config_quantum_II['batch_size'], shuffle=True, num_workers=0
)
val_loader_qII = DataLoader(
```

```
    JetDataset(X_val, mask_val, y_val),
    batch_size=config_quantum_II['batch_size']
)
train_eval_loader_qII = DataLoader(
    JetDataset(X_train_qII, mask_train_qII, y_train_qII),
    batch_size=config_quantum_II['batch_size']
)

quantum_encoder_II = HybridEncoder(
    in_channels=4,
    embed_dim=32,
    use_quantum=True,
    n_qubits=config_quantum_II['n_qubits'],
    n_qlayers=config_quantum_II['n_qlayers']
).to(device)

n_params = sum(p.numel() for p in quantum_encoder_II.parameters())
n_q_params = config_quantum_II['n_qlayers'] * config_quantum_II['n_qubits']
print(f"Quantum encoder (II): {n_params:,} parameters ({n_q_params} quantum
print(f"Config: {config_quantum_II['n_qubits']} qubits, {config_quantum_II['
print(f"Training on {n_quantum_samples_II} samples")
print(f"Batch size: {config_quantum_II['batch_size']}, Epochs: {config_quant

t_start = time.time()

quantum_encoder_II, quantum_II_best_auc, quantum_II_history = train_contrast
    quantum_encoder_II, train_loader_qII, val_loader_qII, train_eval_loader_
    config_quantum_II, device
)

t_elapsed = time.time() - t_start
print(f"\nQuantum (II) best validation AUC: {quantum_II_best_auc:.4f}")
print(f"Total training time: {t_elapsed/60:.1f} minutes")

# Evaluate
train_emb_qII, train_lab_qII = extract_embeddings(quantum_encoder_II, train_
test_emb_qII, test_lab_qII = extract_embeddings(quantum_encoder_II, test_loa
quantum_II_test_acc, quantum_II_test_auc = linear_probe(train_emb_qII, train

# Results table
n_q = config_quantum['n_qubits']
n_l = config_quantum['n_qlayers']
n_qII = config_quantum_II['n_qubits']
n_lII = config_quantum_II['n_qlayers']

print(f"\n{'Model':<55} {'Test Acc':<12} {'Test AUC':<12}")
print(f"{'PointVector Classical (80k samples, 10 epochs)':<55} {classical_te
print(f"{'PointVector + {0}q {1}l VQC (10k samples, 10 epochs)'.format(n_q,
print(f"{'PointVector + {0}q {1}l VQC (40k samples, 10 epochs)'.format(n_qII

# ROC plot comparing all three
fig, ax = plt.subplots(figsize=(8, 6))

clf_c = LogisticRegression(max_iter=1000).fit(train_emb, train_lab)
fpr_c, tpr_c, _ = roc_curve(test_lab, clf_c.predict_proba(test_emb_classical
ax.plot(fpr_c, tpr_c, lw=2, label=f'Classical (AUC={classical_test_auc:.4f})
```

```
clf_q = LogisticRegression(max_iter=1000).fit(train_emb_q, train_lab_q)
fpr_q, tpr_q, _ = roc_curve(test_lab, clf_q.predict_proba(test_emb_quantum)[
ax.plot(fpr_q, tpr_q, lw=2, label=f'Quantum {n_q}q {n_l}l, 10k samples (AUC=

clf_qII = LogisticRegression(max_iter=1000).fit(train_emb_qII, train_lab_qII
fpr_qII, tpr_qII, _ = roc_curve(test_lab_qII, clf_qII.predict_proba(test_emb
ax.plot(fpr_qII, tpr_qII, lw=2, label=f'Quantum {n_qII}q {n_lII}l, 40k sampl

ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC: PointVector Classical vs Quantum Hybrid (ZXZ, Amplitude E
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_pointvector_II_comparison.png', dpi=150)
plt.show()
```

```
Quantum encoder (II): 790,774 parameters (54 quantum gate params)
Config: 6 qubits, 3 layers, ZXZ rotation, amplitude embedding
Training on 40000 samples
Batch size: 32, Epochs: 10, LR: 0.0005
Epoch   1 | Loss: 2.9781 | Val Acc: 0.7623 | Val AUC: 0.8346
Epoch   5 | Loss: 2.5868 | Val Acc: 0.7610 | Val AUC: 0.8414
Epoch  10 | Loss: 2.5309 | Val Acc: 0.7762 | Val AUC: 0.8524

Quantum (II) best validation AUC: 0.8524
Total training time: 43.1 minutes
```

Model	Test Acc	Test AU
C		
PointVector Classical (80k samples, 10 epochs)	0.7831	0.8624
PointVector + 6q 2l VQC (10k samples, 10 epochs)	0.7592	0.8384
PointVector + 6q 3l VQC (40k samples, 10 epochs)	0.7738	0.8551



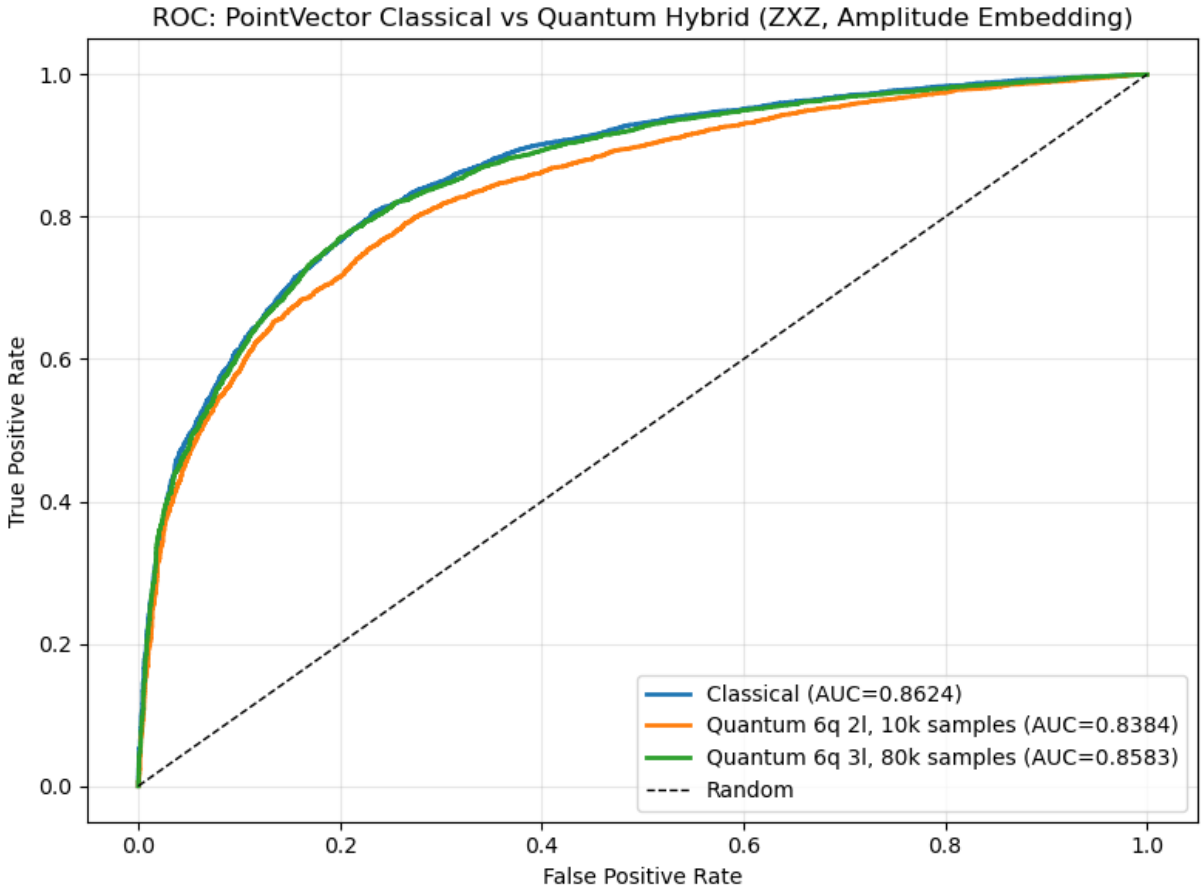
In [27]:
```
# Full dataset quantum training - 6 qubits, 3 layers
import time

config_quantum_full = {
    'model_name': 'pointvector_quantum_amplitude_fulldata',
    'batch_size': 32,
    'lr': 5e-4,
    'n_epochs': 10,
    'temperature': 0.5,
    'use_quantum': True,
    'n_qubits': 6,
    'n_qlayers': 3
}

n_quantum_samples_full = len(X_train)

train_loader_qf = DataLoader(
    JetContrastiveDataset(X_train, mask_train, y_train, aug),
    batch_size=config_quantum_full['batch_size'], shuffle=True, num_workers=
)
val_loader_qf = DataLoader(
    JetDataset(X_val, mask_val, y_val),
    batch_size=config_quantum_full['batch_size']
)
train_eval_loader_qf = DataLoader(
    JetDataset(X_train, mask_train, y_train),
    batch_size=config_quantum_full['batch_size']
)
```

```
quantum_encoder_full = HybridEncoder(
    in_channels=4,
    embed_dim=32,
    use_quantum=True,
    n_qubits=config_quantum_full['n_qubits'],
    n_qlayers=config_quantum_full['n_qlayers']
).to(device)

n_params = sum(p.numel() for p in quantum_encoder_full.parameters())
n_q_params = config_quantum_full['n_qlayers'] * config_quantum_full['n_qubit
print(f"Quantum encoder (full): {n_params:,} parameters ({n_q_params} quantu
print(f"Config: {config_quantum_full['n_qubits']} qubits, {config_quantum_fu
print(f"Training on full dataset: {n_quantum_samples_full} samples")
print(f"Batch size: {config_quantum_full['batch_size']}, Epochs: {config_qua

t_start = time.time()

quantum_encoder_full, quantum_full_best_auc, quantum_full_history = train_co
    quantum_encoder_full, train_loader_qf, val_loader_qf, train_eval_loader_
    config_quantum_full, device
)

t_elapsed = time.time() - t_start
print(f"\nQuantum (full) best validation AUC: {quantum_full_best_auc:.4f}")
print(f"Total training time: {t_elapsed/60:.1f} minutes")

# Evaluate
train_emb_qf, train_lab_qf = extract_embeddings(quantum_encoder_full, train_
test_emb_qf, test_lab_qf = extract_embeddings(quantum_encoder_full, test_loa
quantum_full_test_acc, quantum_full_test_auc = linear_probe(train_emb_qf, tr

# Results table
n_q = config_quantum['n_qubits']
n_l = config_quantum['n_qlayers']
n_qf = config_quantum_full['n_qubits']
n_lf = config_quantum_full['n_qlayers']

print(f"\n{'Model':<50} {'Test Acc':<12} {'Test AUC':<12}")
print(f"{'PointVector Classical (80k samples, 10 epochs)':<50} {classical_te
print(f"{'PointVector + {0}q {1}l VQC (10k samples, 10 epochs)'.format(n_q,
print(f"{'PointVector + {0}q {1}l VQC (80k samples, 10 epochs)'.format(n_qf,

# ROC plot comparing all three
fig, ax = plt.subplots(figsize=(8, 6))

clf_c = LogisticRegression(max_iter=1000).fit(train_emb, train_lab)
fpr_c, tpr_c, _ = roc_curve(test_lab, clf_c.predict_proba(test_emb_classical
ax.plot(fpr_c, tpr_c, lw=2, label=f'Classical (AUC={classical_test_auc:.4f})

clf_q = LogisticRegression(max_iter=1000).fit(train_emb_q, train_lab_q)
fpr_q, tpr_q, _ = roc_curve(test_lab, clf_q.predict_proba(test_emb_quantum)[
ax.plot(fpr_q, tpr_q, lw=2, label=f'Quantum {n_q}q {n_l}l, 10k samples (AUC=

clf_qf = LogisticRegression(max_iter=1000).fit(train_emb_qf, train_lab_qf)
fpr_qf, tpr_qf, _ = roc_curve(test_lab_qf, clf_qf.predict_proba(test_emb_qf)
```

```
ax.plot(fpr_qf, tpr_qf, lw=2, label=f'Quantum {n_qf}q {n_lf}l, 80k samples (

ax.plot([0, 1], [0, 1], 'k--', lw=1, label='Random')
ax.set_xlabel('False Positive Rate')
ax.set_ylabel('True Positive Rate')
ax.set_title('ROC: PointVector Classical vs Quantum Hybrid (ZXZ, Amplitude E
ax.legend(loc='lower right')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('roc_pointvector_full_comparison.png', dpi=150)
plt.show()
```

```
Quantum encoder (full): 790,774 parameters (54 quantum gate params)
Config: 6 qubits, 3 layers, ZXZ rotation, amplitude embedding
Training on full dataset: 80000 samples
Batch size: 32, Epochs: 10, LR: 0.0005
Epoch   1 | Loss: 2.9156 | Val Acc: 0.7707 | Val AUC: 0.8415
Epoch   5 | Loss: 2.5575 | Val Acc: 0.7818 | Val AUC: 0.8548
Epoch  10 | Loss: 2.5126 | Val Acc: 0.7839 | Val AUC: 0.8573

Quantum (full) best validation AUC: 0.8573
Total training time: 81.3 minutes
```

Model	Test Acc	Test AUC
PointVector Classical (80k samples, 10 epochs)	0.7831	0.8624
PointVector + 6q 2l VQC (10k samples, 10 epochs)	0.7592	0.8384
PointVector + 6q 3l VQC (80k samples, 10 epochs)	0.7830	0.8583



In [ ]: